Ali Alemami

Systems & Backend Developer

Amman, Jordan | ali.alemami01@gmail.com | +962 780 020 203 | github.com/ali-alemami | ali-alemami.github.io | linkedin.com/in/ali-alemami

Computer Science student specializing in backend & low-level systems engineering, with hands-on experience in **C/C++** (POSIX APIs, multithreading, memory management) and **C#/.NET** (3-tier architecture, SQL Server).

TECHNICAL SKILLS

Programming Languages:	C (POSIX/C99), C++ (98/11/17), C# (.NET), SQL (T-SQL), Bash
Systems & OS Concepts:	POSIX APIs, Multithreading (pthreads), Concurrency & Mutexes, Sockets / epoll, Process Control & IPC, Memory (Valgrind/ASan)
CS Fundamentals:	Data Structures & Algorithms, OOP & Design Patterns, 3-Tier Architecture, Computer Networks
Tools & Infrastructure:	SQL Server, ADO.NET, Docker, Docker Compose, NGINX, Git, Linux, GDB, Makefiles
Languages (Spoken):	Arabic (Native), English (Fluent / Professional)

EXPERIENCE

42 Amman — Full-time intensive program, peer-reviewed project development — 2025 – Present

Structured, deadline-driven software engineering program — 22/24 Common Core projects complete

- **Multithreading & Concurrency** (C — philo): Engineered a race-free simulation of Dijkstra's Dining Philosophers using pthreads and mutex locks; built async monitoring routines with microsecond-precision timers preventing thread starvation and deadlocks.
- **HTTP Web Server** (C++98 — webserv): Building a non-blocking HTTP/1.1 server using epoll I/O multiplexing, with stateful request parsing, NGINX-style config parsing, and CGI execution.
- **DevOps & Container Infrastructure** (inception): Architected a multi-container infrastructure with Docker Compose on Alpine Linux; isolated containers for NGINX (TLS/SSL), WordPress, and MariaDB with bind mounts and internal bridge networking.
- **Unix Shell & AST Engine** (C — minishell): Implemented a POSIX-compliant shell with an AST parser enforcing operator precedence, nested subshell forks, and quote-aware tokenization; zero leaks via Valgrind.
- **3D Graphics & Raytracing Engine** (C — mini_rt): Built a 3D raytracing renderer from scratch in C, implementing ray-surface intersection math and Phong reflection models (ambient, diffuse, specular) rendered via MiniLibX.
- **C++ Modules & Ford-Johnson Algorithm** (C++ — cpps): Completed 42 C++ Modules 00–09 (OOP, polymorphism, templates, STL); implemented Ford-Johnson Merge-Insert Sort optimizing comparisons via Jacobsthal sequence pairing.
- **Foundations & Algorithms:** push_swap (bitwise radix sort across two stacks), pipex (multi-pipe fork/dup2), fract-ol (fractal rendering), minitalk (UNIX signal IPC), libft, ft_printf, get_next_line.
- **Peer Review & Collaboration:** Conducted 50+ technical evaluations and collaborated in pair-programming teams (minishell, mini_rt).

SOFTWARE DEVELOPMENT & DATABASE PROJECTS

Programming Advices roadmap (Dr. Mohammed Abu-Hadhoud) — 17/24 courses completed in C#/.NET, SQL Server, and C++

- **Bank & ATM CLI System** (C++ — Bank-and-ATM-System-CPP): Built a terminal-based banking and ATM system in C++ with bitmask user permissions, transaction audit logging, and file-based persistence.
- **Contacts Management App** (C#, ADO.NET, SQL Server — Contacts-App-3Tier): Developed a 3-tier desktop application separating UI, business logic, and data access layers; used parameterized T-SQL queries to prevent SQL injection.
- **Relational Database Schemas & SQL** (SQL Server — Programming-Advices-Core): Designed 5 relational database schemas (Medical Clinic Booking, E-Commerce Store, HR/Payroll, Vehicle Registry, Employee Analytics) with normalized keys and multi-table JOIN queries.
- **Algorithms & Problem Solving** (C++ — 225+ Problems): Implemented solutions across 5 difficulty levels covering data structures, matrix operations, string parsing, and combinatorial algorithms.
- **Windows Forms Projects** (C#, .NET — CSharp-WinForms-Collection): Built 10+ desktop utility and game apps in C#, focusing on UI event handling and object-oriented design.

EDUCATION

The Hashemite University — BSc in Computer Science — 2022 – 2026 (Expected)